COL775 Assignment 1 - Part 2
Neural Machine Translation and Cross-Lingual Transfer

Yash Bansal (2022CS51133)

March 27, 2026

Contents

1 Introduction

This part of the assignment implements and evaluates a Neural Machine Translation (NMT) system for translating English sentences into Hindi, and subsequently investigates cross-lingual transfer to Marathi. We implement a sequence-to-sequence architecture combining various encoder designs with an LSTM-based decoder with attention.

1.1 Dataset

We work with two translation datasets. The English–Hindi dataset contains 58,494 training samples, 3,242 development samples, and 2,256 test samples. The English–Marathi dataset contains 18,979 training samples, 1,055 development samples, and 556 test samples. All models are evaluated using BLEU, chrF, and TER scores computed via the `sacrebleu` library.

1.2 Experimental Overview

We conduct the following experiments:

1. **Seq2Seq with GloVe:** A bidirectional LSTM encoder with GloVe embeddings and an LSTM decoder, trained without attention.
2. **Seq2Seq+Attention with GloVe:** The same architecture extended with an attention mechanism over encoder hidden states.
3. **Seq2Seq+Attention with Frozen BERT:** A pre-trained BERT-base-cased encoder (frozen) paired with an LSTM decoder with attention.
4. **Seq2Seq+Attention with Fine-tuned BERT:** Same as above but with the BERT encoder fine-tuned end-to-end along with the decoder.
5. **Beam Search Decoding:** Evaluation of the fine-tuned BERT model using beam search with beam sizes $\{5, 10, 20\}$, compared against greedy decoding.
6. **Teacher Forcing Schedule:** Training the fine-tuned BERT model with an inverse sigmoid decay schedule for teacher forcing instead of a fixed probability.
7. **Cross-Lingual Transfer:** Fine-tuning the English–Hindi model on English–Marathi data and comparing against a model trained from scratch on Marathi.

All models use teacher forcing with a fixed ratio of 0.6 during training and 0 during inference, unless otherwise specified. The decoder uses a custom BPE tokenizer trained on the target-side corpus.

1.3 Model Availability

Trained model weights are available at:
`https://drive.google.com/drive/folders/1i7HnvshFrOqIdK999mz4JvDvrzuQHYt5?usp=sharing`

2 Implementation and Design Choices

2.1 Architecture Overview

All models follow a sequence-to-sequence framework consisting of an encoder and an LSTM-based decoder. The encoder varies across models, while the decoder architecture remains consistent. All models are implemented from scratch as `torch.nn.Module` subclasses.

2.2 Encoders

2.2.1 BiLSTM Encoder (Models 1 and 2)

The BiLSTM encoder uses 100-dimensional GloVe embeddings (`glove.6B.100d`) to initialize the source embedding layer. The embeddings are loaded from `torchtext` and words not found in GloVe are initialized with random normal noise ($\mu = 0$, $\sigma = 0.1$). The encoder uses a single-layer bidirectional LSTM with hidden dimension 256, producing outputs of dimension 512. Since the decoder is unidirectional, the final hidden and cell states from both directions are concatenated and projected back to 256 dimensions using a linear layer followed by LayerNorm and tanh activation. This ensures the decoder is initialized with a compact, well-normalized summary of the source sentence.

2.2.2 BERT Encoder (Models 3 and 4)

The BERT encoder uses `bert-base-cased` from HuggingFace. The full sequence of last hidden states (dimension 768) is projected to $512 = 2 \times 256$ dimensions using a linear layer with LayerNorm to serve as encoder outputs for attention. The `[CLS]` token representation is separately projected to 256 dimensions to initialize the decoder hidden and cell states, again with LayerNorm and tanh.

For Model 3, all BERT parameters are frozen and only the projection layers and decoder are trained. For Model 4, BERT is fine-tuned end-to-end with a lower learning rate (10^{-4}) than the rest of the network (10^{-3}) to avoid catastrophic forgetting of pre-trained representations.

2.3 Decoders

2.3.1 Vanilla Decoder (Model 1)

The vanilla decoder uses a single-layer LSTM that receives the concatenation of the target token embedding and the mean-pooled encoder output as input at each step. The output is projected to the target vocabulary via a linear layer. This decoder does not use attention and relies entirely on the encoder's compressed representation.

2.3.2 Attention Decoder (Models 2, 3, 4)

The attention decoder uses Bahdanau-style additive attention. At each decoding step, the attention module computes a weighted sum of encoder outputs conditioned on the current decoder hidden state. The attention scores are computed as:

$$e_{t,i} = \mathbf{v}^\top \tanh\left(\text{LayerNorm}\left(W\left[\mathbf{h}_{t-1}; \mathbf{e}_i\right]\right)\right) \tag{1}$$

where $\mathbf{h}_{t-1}$ is the decoder hidden state and $\mathbf{e}_i$ is the i-th encoder output. Padding positions are masked out by setting their scores to $-\infty$ before the softmax. The context vector is concatenated with the LSTM output before the final projection to the vocabulary, giving the model direct access to both the attended source context and the decoded hidden state when making predictions.

2.4 Tokenization

2.4.1 Source Tokenization

For GloVe-based models, source sentences are tokenized using a simple regex pattern that extracts alphanumeric tokens and punctuation, lowercased. All words with frequency ≥ 1 in the training corpus are included in the vocabulary. For BERT-based models, the standard `bert-base-cased` WordPiece tokenizer is used with a maximum length of 512 tokens.

2.4.2 Target Tokenization

All models use a custom Byte Pair Encoding (BPE) tokenizer trained on the target-side training corpus using the `tokenizers` library, with a vocabulary size of 10,000 and minimum frequency of 1. Special tokens `<pad>`, `<unk>`, `<sos>`, and `<eos>` are included. A separate BPE tokenizer is trained for each target language (Hindi and Marathi).

2.5 Training Configuration

The following hyperparameters are used consistently across all models unless otherwise specified:

Table 1: Training hyperparameters for all models

Hyperparameter	**Value**
Embedding dimension	100
Hidden dimension	256
Number of LSTM layers	1
Target vocabulary size (BPE)	10,000
Training batch size	16
Validation/Test batch size	64
Epochs	30
Optimizer	AdamW
Learning rate (non-BERT)	10^{-3}
Learning rate (BERT parameters)	10^{-4}
LR scheduler	Cosine Annealing ($\eta_{\min} = 10^{-4}$)
Gradient clipping	5.0
Teacher forcing ratio (fixed)	0.6 (train), 0.0 (val/test)
Loss function	Cross-Entropy (ignoring `<pad>`)

Model checkpointing is based on best validation BLEU score, and the best checkpoint is used for final evaluation on all splits.

2.6 Hyperparameter Tuning

The hyperparameter search space was heavily constrained by the available compute resources. All models were trained on a Kaggle P100 GPU (16 GB VRAM) with a time budget of 10–12 hours per run. The configuration was chosen such that the slowest model (Model 4, fine-tuned BERT) could complete 30 epochs within this budget.

- **Architecture size:** The number of LSTM layers is fixed to 1 and the hidden dimension to 256, which is the largest configuration that allows Model 4 to train within the memory and time constraints on the Kaggle P100.

- **Batch size:** A training batch size of 16 was chosen as the largest value that fits Model 4 in GPU memory given the BERT encoder overhead.
- **Target vocabulary size:** The BPE vocabulary size is set to 10,000 based on the compression ratio of the Hindi and Marathi target corpora. The tokenizer was observed to converge well at this vocabulary size, providing good subword coverage without unnecessary overhead.
- **Learning rates:** Standard learning rates for the Adam optimizer family are used (10^{-3} for non-BERT parameters, 10^{-4} for BERT parameters), which work reasonably well without requiring further tuning.

2.7 Teacher Forcing

Teacher forcing is applied at the batch level: for each batch, a single Bernoulli draw with probability p determines whether ground-truth tokens or model predictions are used as decoder inputs for the entire batch. During validation and inference, teacher forcing is always set to 0 so the model generates autoregressively from its own predictions. For the inverse sigmoid schedule experiment, the teacher forcing probability at epoch t is:

$$p(t) = \frac{k}{k + \exp(t/k)}, \quad k = 8 \tag{2}$$

This starts close to $p \approx 1$ in early epochs and smoothly decays toward $p \approx 0$, providing strong supervision early in training while progressively forcing the model to rely on its own outputs.

2.8 Beam Search Decoding

Beam search is implemented from scratch in `evaluate.py`. At each decoding step, the top-B candidate sequences are maintained based on cumulative log-probability scores. Completed beams (those that have generated `<eos>`) are frozen in place. Final beam selection uses length-normalized scores to avoid a bias toward shorter sequences:

$$\text{score}_{\text{norm}} = \frac{\log P(\mathbf{y})}{\text{len}(\mathbf{y})} \tag{3}$$

Beam search is evaluated at beam sizes $\{1, 5, 10, 20\}$, where beam size 1 corresponds to greedy decoding. Average decoding time per sentence is recorded for each beam size to quantify the computational trade-off.

2.9 Cross-Lingual Transfer

For the Marathi transfer experiment, the fine-tuned English–Hindi model checkpoint is loaded and the Marathi BPE tokenizer is trained fresh on the Marathi target corpus. Since the target vocabulary changes between Hindi and Marathi, the embedding layer (`decoder.embeddings.weight`) and output projection (`decoder.fc_out.weight`, `decoder.fc_out.bias`) are excluded from weight loading and randomly re-initialized. All other model weights, including the BERT encoder and attention decoder LSTM, are transferred and fine-tuned on the English–Marathi training data for the same number of epochs as the from-scratch baseline to ensure a fair comparison.

3 English–Hindi Translation Results

We train all four model variants on the English–Hindi dataset using the hyperparameters described above, with a fixed teacher forcing ratio of 0.6 during training and 0.0 during inference. The best checkpoint for each model is selected based on validation BLEU score.

3.1 Training Curves

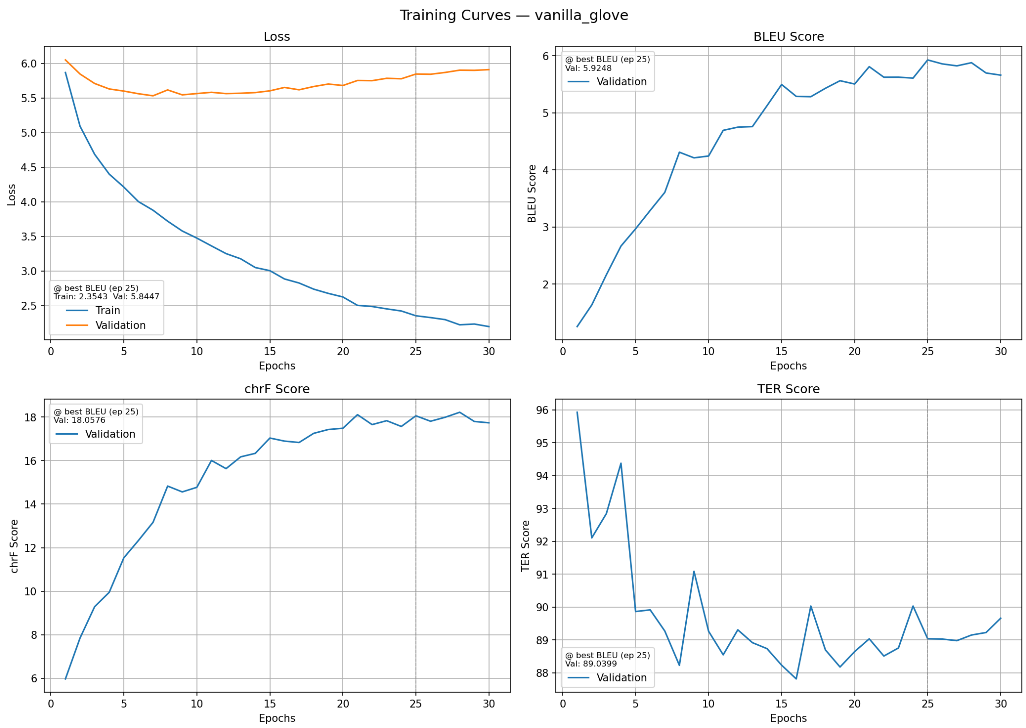

Figure 1: Training curves for Model 1 (Seq2Seq with GloVe, no attention).

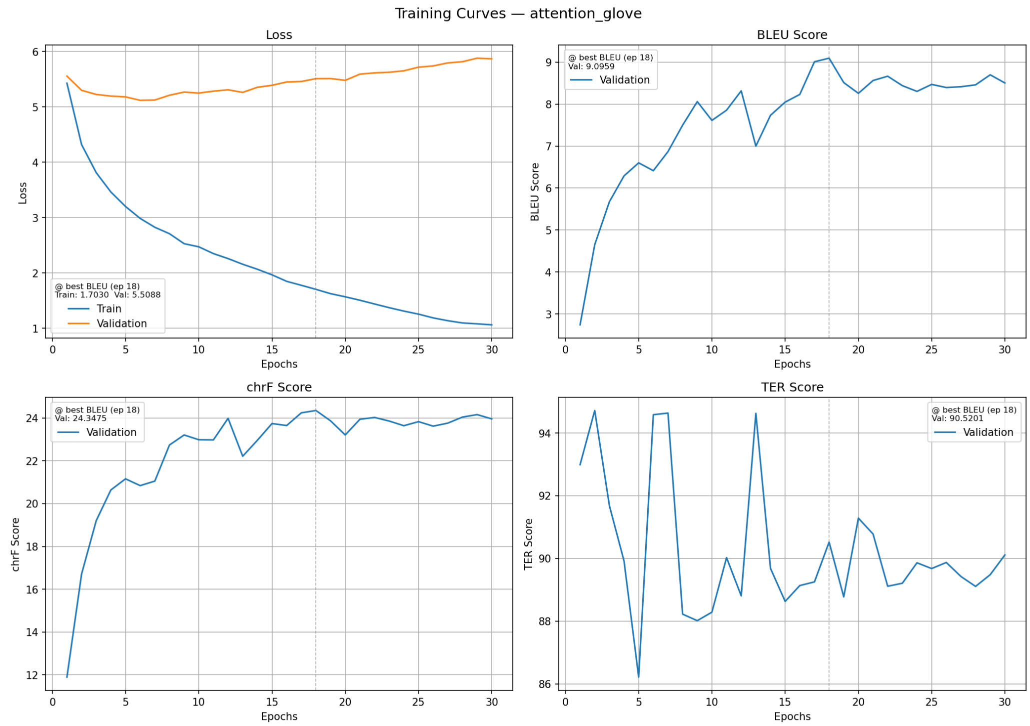

Figure 2: Training curves for Model 2 (Seq2Seq+Attention with GloVe).

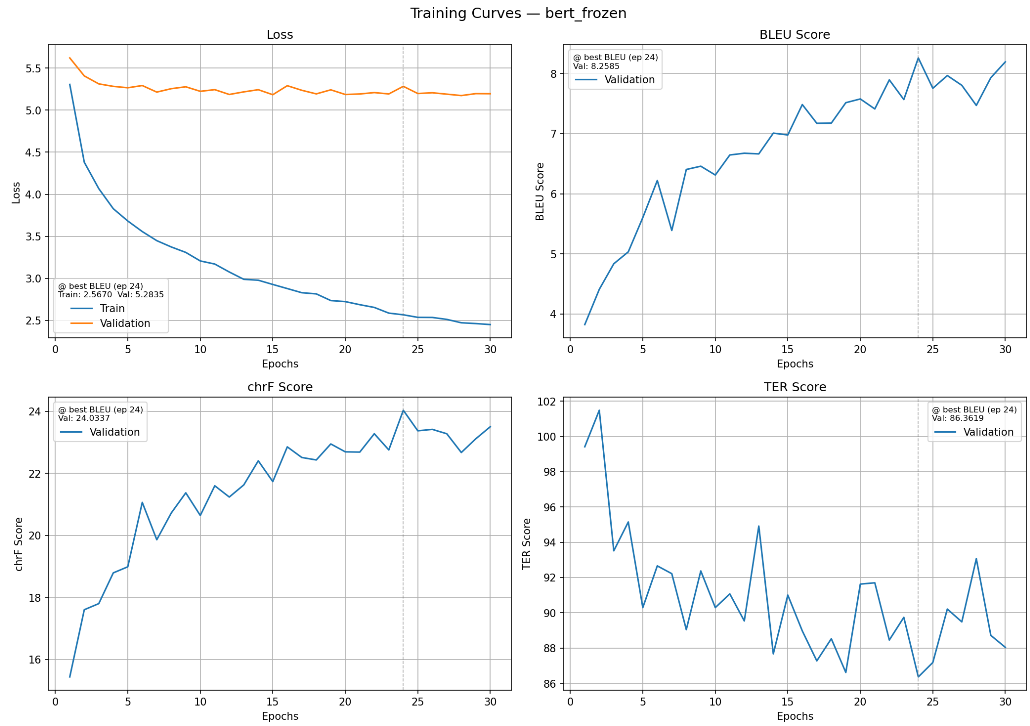

Figure 3: Training curves for Model 3 (Seq2Seq+Attention with frozen BERT).

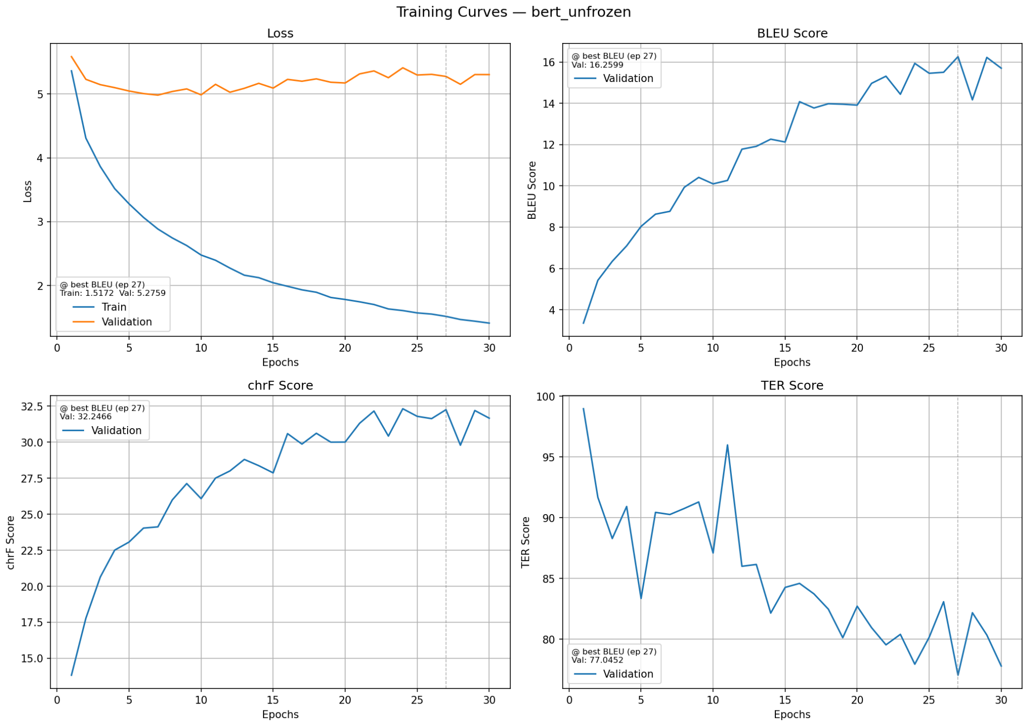

Figure 4: Training curves for Model 4 (Seq2Seq+Attention with fine-tuned BERT).

3.2 Quantitative Results

Table 2 reports the final test set scores for all four models using greedy decoding.

Table 2: Test set results for all models on English–Hindi translation (greedy decoding)

Model	**BLEU** ↑	**chrF** ↑	**TER** ↓
Model 1: Seq2Seq + GloVe (no attention)	5.87	18.48	88.80
Model 2: Seq2Seq + Attention + GloVe	9.46	25.05	85.08
Model 3: Seq2Seq + Attention + Frozen BERT	8.23	24.06	83.81
Model 4: Seq2Seq + Attention + Fine-tuned BERT	**17.19**	**33.48**	**74.32**

3.3 Sample Translations

Table 3 shows representative sample translations from each model for qualitative comparison.

Table 3: Sample translations from all models on the English–Hindi test set

Source	**Reference**	**Model Predictions**
Short / simple sentence		
playing field size	खेल के मैदान का आकार	M1: खेल क्षेत्र आकार M2: खेल के क्षेत्र का आकार M3: खेल क्षेत्र का आकार M4: खेल मैदान का आकार
Medium sentence		
File exists but is not a regular file.	फ़ाइल मौजूद है लेकिन एक सामान्य फ़ाइल नहीं है .	M1: फ़ाइल (□) मौजूद नहीं है M2: फ़ाइल मौजूद है लेकिन M3: फ़ाइल मौजूद है लेकिन सामान्य नहीं है M4: फ़ाइल मौजूद है , एक नियमित फ़ाइल नहीं .
Long / complex sentence (failure case)		
braille the falling letters	ला पता अक्षर भरे	M1: मा मा मा मा मा क M2: ज ने का चि प स प्ला ल M3: वे ल शब्द को M4: पॉ ल टे लम ◌ारी

3.4 Analysis

- **Effect of attention:** Model 2 significantly outperforms Model 1 across all metrics (BLEU: 9.46 vs 5.87, chrF: 25.05 vs 18.48), confirming that the attention mechanism provides a substantial benefit by allowing the decoder to selectively focus on relevant source positions rather than relying solely on the compressed encoder summary.

- **Frozen vs fine-tuned BERT:** Model 3 (frozen BERT, BLEU 8.23) performs slightly worse than Model 2 (GloVe+attention, BLEU 9.46), suggesting that frozen BERT representations, while rich, are not directly optimized for the translation task and the projection layers alone are insufficient to bridge this gap. Model 4, where BERT is fine-tuned end-to-end, achieves by far the best results (BLEU 17.19, chrF 33.48, TER 74.32), demonstrating the importance of adapting pre-trained representations to the downstream translation task.

- **Validation loss behavior across models:** The training curves reveal interesting differences in validation loss dynamics. Model 1 (Figure 1) shows a nearly flat validation loss throughout training (≈ 5.6), suggesting the model quickly saturates its capacity without attention. Model 2 (Figure 2) shows the validation loss declining initially before gradually creeping upward from around epoch 8–10. Model 3 (Figure 3) is the most distinctive: the validation loss remains remarkably stable at ≈ 5.2 across all 30 epochs, reflecting the fact that the frozen BERT encoder does not adapt during training and the remaining parameters (projection layers, decoder) converge quickly to a stable regime. Model 4 (Figure 4) shows a gentle upward drift in validation loss (from ≈ 5.0 to ≈ 5.3) while the validation BLEU continues to improve up to epoch 27 — a classic divergence between teacher-forced loss and autoregressive generation quality in seq2seq training.

- **TER scores:** GloVe-based models have high TER scores (> 85), indicating many edits are needed to convert predictions to references. Model 4 reduces this substantially to 74.32, though there is still considerable room for improvement.

- **Repetition in Model 1:** The vanilla model without attention frequently produces degenerate outputs with repetitive tokens (e.g., 'मा मा मा मा मा'), a known failure mode of seq2seq models without attention when the encoder hidden state loses track of the source content over long sequences.

- **High BLEU scores on short sentences:** Several sentences achieve perfect BLEU scores (100.00) across all models. Examining these examples reveals that they are very short phrases (1–3 tokens) that likely appear verbatim in the training data as well, since the dataset is drawn from overlapping sources. This inflates per-sentence BLEU on short segments and should be interpreted with caution.

- **Consistent failure cases:** Some sentences fail across all models (e.g., "braille the falling letters", "LyX"), suggesting these are either rare domain-specific terms, heavily transliterated phrases, or sentences whose references in the dataset are themselves noisy or inconsistent.

4 Beam Search Decoding

We evaluate the fine-tuned BERT model (Model 4) using beam search decoding with beam sizes $\{1, 5, 10, 20\}$, where beam size 1 corresponds to standard greedy decoding. All other settings remain identical to the training configuration.

4.1 Quantitative Results

Table 4: Beam search results for Model 4 (fine-tuned BERT) on the English–Hindi test set

Beam Size	BLEU ↑	chrF ↑	TER ↓	Avg Time/Sent (ms)
1 (Greedy)	17.19	33.48	74.32	1.93
5	23.11	39.10	69.17	3.08
10	24.65	40.64	67.90	3.22
20	25.50	41.56	67.29	4.57

4.2 Sample Translations

Table 5 shows representative translations at different beam sizes for qualitative comparison.

Table 5: Sample translations at different beam sizes (Model 4)

Source	Reference	Predictions by beam size
Medium sentence — improvement with beam search		
There is in the heavens your sustenance, and whatever has been promised you.	और तुम्हारी रोज़ी और जिस चीज़ का तुमसे वायदा किया जाता है आसमान में है	B=1: रात को रात तमाम (दुनिया के) मुक़र्रर भर मुक़र्रर हैं B=10: और आकाश मे ही तुम्हारी रोज़ी है और वह चीज़ भी तुमसे वादा किया जाता है
Consistent failure across beam sizes		
Braille the falling letters	ला पता अक्षर भरे	B=1: पॉ ल टे लम ◌ारी B=5: पॉ ल टे लम की लो B=10: पॉ ल टे लम की लो
Short sentence		
How was that?	वह कैसा था ?	B=1: वह कैसे था ? B=5: वह कैसे था ? B=10: वह कैसे था ?

4.3 Analysis

- **Consistent improvement with beam size:** Increasing the beam size from 1 to 5 yields a large improvement in BLEU (17.19 → 23.11, +5.92) and chrF (33.48 → 39.10), with a corresponding drop in TER (74.32 → 69.17). This confirms that greedy decoding is suboptimal and that exploring a wider search space finds substantially better translations.
- **Diminishing returns beyond beam size 5:** The improvement from beam size 5 to 10 is smaller (BLEU: +1.54, chrF: +1.54), and from 10 to 20 smaller still (BLEU: +0.85, chrF: +0.92), suggesting diminishing returns as the beam size grows. The best candidates are largely already captured by beam size 5, and further expansion recovers only marginal gains.

- **Decoding time trade-off:** Greedy decoding averages 1.93 ms per sentence. Beam size 5 increases this to 3.08 ms ($\approx 1.6\times$ slower) and beam size 10 to 3.22 ms ($\approx 1.7\times$ slower). Beam size 20 reaches 4.57 ms ($\approx 2.4\times$ slower). Notably, the jump from beam size 5 to 10 adds only 0.14 ms in latency while yielding a BLEU gain of 1.54, making beam size 10 an attractive operating point. Beam size 20 offers the best translation quality overall but at higher cost with comparatively smaller quality gains over beam size 10.
- **Qualitative improvement:** For medium and long sentences, beam search produces noticeably more fluent and complete translations compared to greedy decoding, which sometimes produces truncated or repetitive outputs. Short sentences show less difference between beam sizes, as greedy decoding is already sufficient to find the correct short sequence.
- **Consistent failure cases:** Some sentences such as "Braille the falling letters" fail at all beam sizes, indicating that the error is not due to search but rather to a fundamental lack of knowledge about the translation of these terms in the model. Wider beam search does not help when the correct translation is not reachable within the model's learned distribution.

5 Effect of Teacher Forcing Schedule

We train Model 4 (fine-tuned BERT) using an inverse sigmoid decay schedule for teacher forcing, as described above, and compare against the fixed teacher forcing probability (0.6) used in the previous section.

5.1 Training Curves

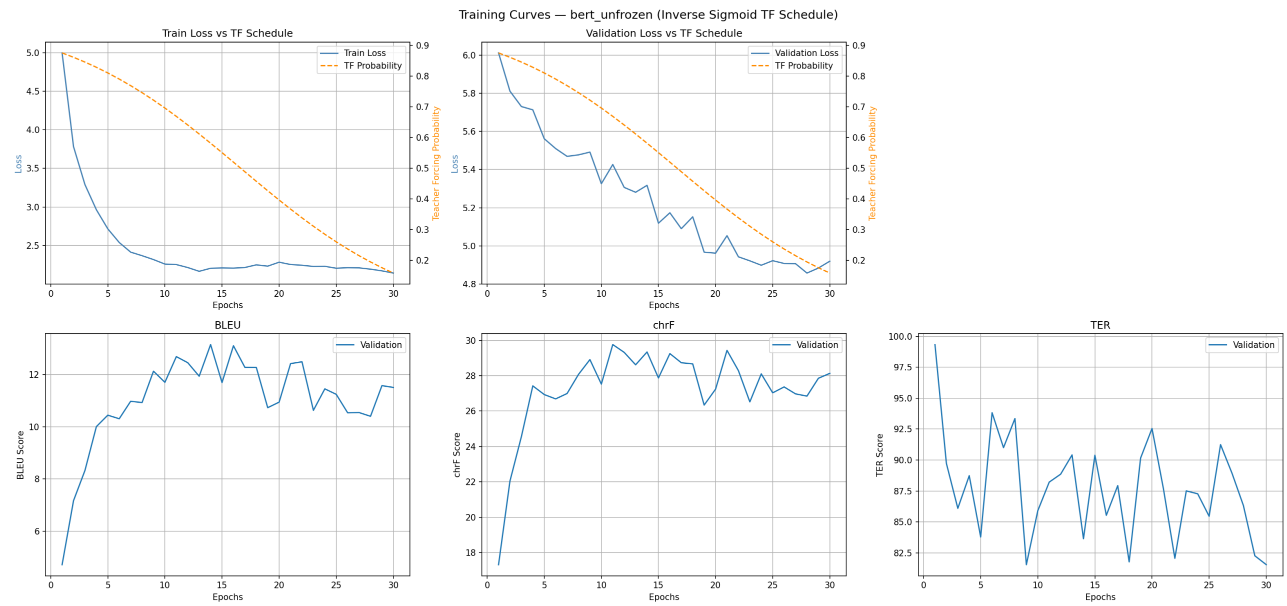

Figure 5: Training curves for Model 4 with inverse sigmoid teacher forcing schedule. The top row shows training loss (left) and validation loss (right), each overlaid with the TF probability on the secondary axis (dashed orange). The bottom row shows validation BLEU, chrF, and TER curves.

5.2 Quantitative Results

Table 6: Test set comparison: fixed teacher forcing (0.6) vs inverse sigmoid schedule

Configuration	**BLEU** ↑	**chrF** ↑	**TER** ↓
Fixed TF (ratio = 0.6)	**17.19**	**33.48**	**74.32**
Inverse Sigmoid TF (k = 8)	14.82	31.23	79.71

5.3 Sample Translations

Table 7: Sample translations from the TF schedule model

Source	Reference	Prediction
I will not be able listen to your story.	मैं सक्षम अपनी कहानी सुना नहीं होगा .	मैं आपकी कहानी को नहीं सुनो नहीं
This is the Truth from your Lord, therefore beware – do not be in doubt.	सत्य तुम्हारे रब की ओर से है । अतः तुम सन्देह करनेवालों में से कदापि न होगा	यह यह सत्य है कि तुम्हारे रब की ओर से हक़ है । अतः तुम सन्देह पड़ पड़ ों से न पड़ ना
Braille the falling letters	ला पता अक्षर भरे	ब्रे ल - पत्र

5.4 Analysis

- **Fixed TF outperforms scheduled TF:** The fixed teacher forcing model achieves BLEU 17.19 vs 14.82 for the scheduled variant, and lower TER (74.32 vs 79.71). The scheduled approach, despite its theoretical motivation, underperforms in this setting.

- **Validation loss behavior:** A key difference between the two models is visible in the validation loss curves. With fixed TF (Figure 4), the validation loss dips slightly in the first few epochs (to ≈ 5.0) before gradually rising to ≈ 5.3 by epoch 30 — a mild upward drift while the validation BLEU continues to improve. This train-val divergence is a known characteristic of seq2seq models: teacher-forced cross-entropy loss and autoregressive BLEU measure different things, so BLEU can keep rising even as loss slowly increases. With the scheduled TF (Figure 5), the validation loss actually *decreases* consistently throughout training (from ≈ 6.0 to ≈ 4.8), because as the TF probability decays toward 0, the training and validation conditions become increasingly aligned (both autoregressive), making the validation loss a more faithful proxy for inference performance.

- **Val BLEU instability under scheduled TF:** The validation BLEU under the scheduled TF peaks around epoch 13–15 (reaching ≈ 13.1) and then starts fluctuating and declining in later epochs. This is because as the TF probability approaches 0, the model is forced to generate entirely from its own predictions during training, which introduces compounding errors when the model is not yet robust enough. This exposure bias at low TF probabilities hurts the model's ability to generalize.

- **Train loss convergence:** The fixed TF model converges to a train loss of ≈ 1.5 by epoch 27, while the scheduled TF model plateaus around ≈ 2.2 from epoch 10 onwards. The scheduled TF model has a harder optimization problem as teacher forcing is withdrawn, resulting in a consistently higher training loss compared to fixed TF.

- **Noisy validation metrics:** The BLEU, chrF, and TER curves under the scheduled TF (Figure 5) are considerably noisier than those under fixed TF. This instability is a direct consequence of the decaying TF probability: as the model transitions from guided to fully autoregressive decoding during training, the generated output quality oscillates before stabilizing.

6 Cross-Lingual Transferability

We investigate whether the knowledge learned by the English–Hindi model (Model 4, fine-tuned BERT) can be transferred to English–Marathi translation. Since Hindi and Marathi share significant linguistic similarities in vocabulary, grammar, script (Devanagari), and sentence structure, the representations learned for Hindi translation may provide a useful initialization for Marathi. We compare two settings: training from scratch on English–Marathi, and fine-tuning the English–Hindi checkpoint on English–Marathi for the same number of epochs.

6.1 Training Curves

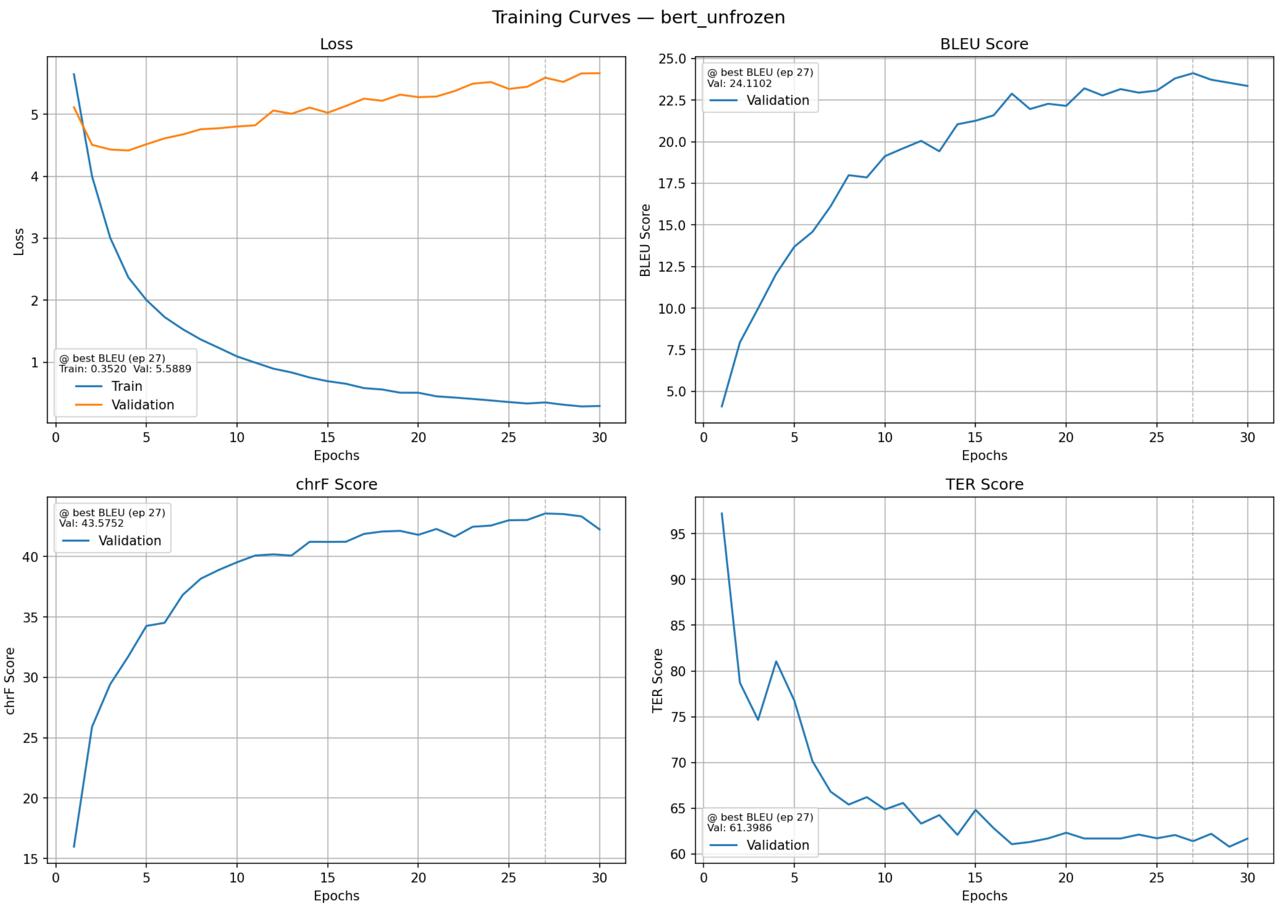

Figure 6: Training curves for the English–Marathi model trained from scratch.

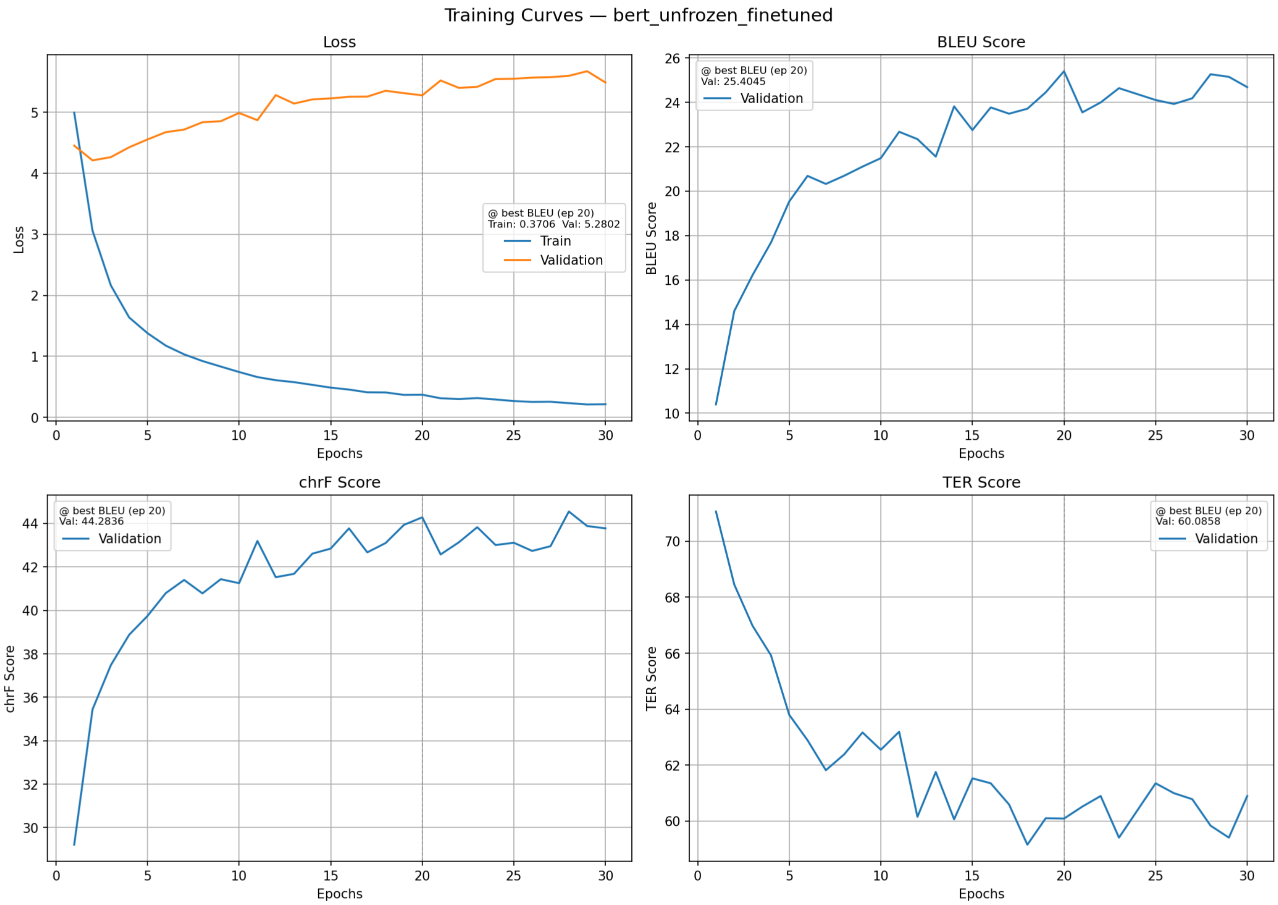

Figure 7: Training curves for the English–Marathi model fine-tuned from the English–Hindi checkpoint.

6.2 Quantitative Results

Table 8: Test set results for English–Marathi translation: from scratch vs cross-lingual transfer

Model	**BLEU** ↑	**chrF** ↑	**TER** ↓
Trained from scratch	22.36	43.64	62.86
Fine-tuned from Hindi model	**23.62**	**44.59**	**62.27**

6.3 Sample Translations

Table 9: Sample translations for English–Marathi: from scratch vs fine-tuned

Source	Reference	Predictions
Simple sentence — both models succeed		
I stayed there for three days.	मी तिथे तीन दिवस राहिले .	Scratch: मी तिथे तीन दिवस राहिले . Fine-tuned: मी तिथे तीन दिवस राहिले .
Medium sentence — fine-tuned slightly better		
Tom doesn't have a bicycle.	टॉमकडे सायकल नाहीये .	Scratch: टॉमकडे सायकल नाहीये . Fine-tuned: टॉमकडे सायकल नाही .
Failure case — both models fail		
Instant	लगेच	Scratch: कार्यपध्दती Fine-tuned: पटकन
Do you know the reason?	तुला कारण माहीत आहे का ?	Scratch: तुला कारण माहीत आहे का ? Fine-tuned: तुम्हाला कारण माहीत आहे का ?

6.4 Analysis

- **Cross-lingual transfer provides modest but consistent improvement:** The fine-tuned model outperforms the from-scratch baseline on all three metrics (BLEU: 23.62 vs 22.36, chrF: 44.59 vs 43.64, TER: 62.27 vs 62.86). While the gains are relatively small in absolute terms, they are consistent across all metrics, confirming that the Hindi representations provide a useful initialization for Marathi.

- **Transfer provides better initialization and faster convergence:** Comparing Figures 6 and 7, the fine-tuned model starts with a noticeably higher initial validation BLEU (≈ 10) compared to the from-scratch model (≈ 5), directly reflecting the benefit of the Hindi encoder weights. Furthermore, the fine-tuned model reaches its best checkpoint at epoch 20, while the from-scratch model does not peak until epoch 27, indicating that cross-lingual transfer leads to faster convergence in addition to a better final result.

- **Severe overfitting in both Marathi models:** Both models exhibit aggressive overfitting: the training loss drops to ≈ 0.35 by the end of training while the validation loss steadily rises from around epoch 5 onwards. This is a direct consequence of the much smaller Marathi training set (18,979 samples vs 58,494 for Hindi), which provides insufficient diversity to prevent memorization at this model capacity. The fine-tuned model shows a slightly less severe increase in validation loss (≈ 5.5 vs ≈ 5.6 for scratch), consistent with a better-regularized starting point from the Hindi checkpoint.

- **Modest gains explained by linguistic similarity:** Hindi and Marathi share the Devanagari script, significant vocabulary overlap, and similar grammatical structure. This means the BERT encoder representations, which are already adapted to Hindi translation, transfer reasonably well to Marathi. The decoder LSTM and attention weights, which capture target-side generation patterns, also benefit from the Hindi initialization since the two languages share many surface forms.

- **Decoder vocabulary re-initialization limits transfer:** Since the target vocabulary differs between Hindi and Marathi, the decoder embedding layer and output projection are re-initialized randomly. This means the generation side of the model starts from scratch for Marathi, limiting the benefit of transfer to the encoder and attention components only. A more sophisticated transfer strategy that leverages the shared Devanagari vocabulary between Hindi and Marathi could potentially yield larger gains.

- **Both models fail on the same cases:** Sentences involving rare proper nouns, transliterated terms, or noisy references (e.g., “Jom”, “Instant”, “Resolver”) fail consistently in both models, suggesting these are dataset-level issues rather than model-level ones.
- **Qualitative differences are subtle:** Looking at the sample translations, both models produce outputs that are largely correct for simple sentences. For the “Do you know the reason?” example, both models produce correct translations but use different levels of formality (तुला vs तुम्हाला), which is a valid variation in Marathi. These subtle differences are not captured well by automatic metrics like BLEU, which may underestimate the true quality gap between the two models.
- **Marathi performance is higher than Hindi:** Both Marathi models (BLEU $\sim$22–24) outperform the best Hindi model (BLEU 17.19 with greedy decoding). This is likely due to the Marathi test set being smaller (556 samples) and possibly less diverse, as well as the fact that Marathi sentences in the dataset tend to be shorter and more straightforward on average.

7 Conclusion

In this work, we implemented and evaluated a suite of Neural Machine Translation models for English–Hindi and English–Marathi translation. The results consistently highlight three key findings.

First, the attention mechanism provides a substantial and reliable benefit over the vanilla seq2seq baseline (BLEU 9.46 vs 5.87), confirming its importance for aligning source and target representations during decoding. Second, fine-tuning the BERT encoder end-to-end is critical: frozen BERT (BLEU 8.23) actually underperforms GloVe+attention, while fine-tuned BERT (BLEU 17.19) far surpasses all other models, demonstrating that pre-trained representations must be adapted to the translation task to be useful. Third, beam search decoding yields substantial gains over greedy decoding at modest computational cost, with beam size 5 offering the best quality-to-latency trade-off and diminishing returns beyond that.

On the training dynamics side, the inverse sigmoid teacher forcing schedule did not improve over a fixed ratio of 0.6 within the 30-epoch budget, as the aggressive decay pushed the model into a fully autoregressive regime before it had sufficient robustness. For cross-lingual transfer, fine-tuning from the Hindi checkpoint yielded consistent improvements over training from scratch on Marathi (BLEU 23.62 vs 22.36), with notably faster convergence, validating the hypothesis that shared script and linguistic structure between Hindi and Marathi enables meaningful transfer.

Overall, the results underscore that the choice of encoder and decoding strategy are the dominant factors in NMT quality, while training schedule and cross-lingual transfer provide secondary but real benefits.